

Computer Science & IT

Data Structure & Programming



Queue

Lecture No. 01

By- Abhishek Sir



Recap of Previous Lecture



Topic

Evaluation of postfix and prefix.

Topic

Topic

Topic

Topic

Topics to be Covered



Topic

Linear queue

Topic

Enqueue

Topic

Dequeue

Topic

Circular Queue

Topic

Challenging
Question

Collect

(%)

$$f(n) = O(g(n))$$

$$g(n) = O(h(n))$$

1. Stack implementation

2. permutation: $C_n = \frac{1}{n+1} {}^{2n}C_n$

3. Expression: precedence — table

Stack: HPU, LPOPU

4. Evaluation: postfix. operand stack left to right

operand₂ = pop(), operand₁ = pop

prefix right to left

operand₁ = pop operand₂ = pop()



Topic : Queue

SBI Branch

SWT



front
Deletion

Last Insertion
rear



Topic : Queue



1



Queue



1

2



Topic : Queue



1

2

3



Queue



1

2

3

4



Queue



Increment
rear
and perform
Insertion
at rear
Index

1

2

3

4

5



Topic : Queue

FIFO



1
front

2

3

4

5

6
rear



Topic : Queue



2
front

3

4

5

5



Topic : Queue

Delete/ process
from front Index
and Increment the
front



front 13

4

5

6



Topic : Queue

1. Queue is Linear data structure
2. Queue work in principle of First in First out
3. Queue is 2 ended data structure

front : deletion / dequeue

rear : Insertion / Enqueue



Topic : Queue



```
# define Max 100  
int arr[Max];  
int front = rear = -1;
```

```
int isEmpty() {  
    if (front == -1)  
        return 1;  
    else  
        return 0;  
}
```

Queue



$f = -1$
 $r = -1$

Insert(1)

0	1	2	3	4

$f = 0$
 $r = 0$

1				
0	1	2	3	4

first Insertion

$r = r + 1$

$arr[r] = data$

we increment front Index

$front = 0$



Queue

$f=0$
 $r=0$

Insert(2)

1				
0	1	2	3	4

$f=0$
 $r=1$

1	2			
0	1	2	3	4

$f=0$
 $r=1$

Insert(3)

1	2			
0	1	2	3	4

$f=0$
 $r=2$

1	2	3		
0	1	2	3	4



Queue

$f=0$
 $r=2$

Insert(4)

1	2	3		
0	1	2	3	4

$f=0$
 $r=3$

1	2	3	4	
0	1	2	3	4

$f=0$
 $r=3$

Insert(5)

1	2	3	4	
0	1	2	3	4

$f=0$
 $r=4$

1	2	3	4	5
0	1	2	3	4

Queue



overflow

$f=0$
 $r=4$

Insert(6)

$f=0$
 $r=4$

1	2	3	4	5
0	1	2	3	4

1	2	3	4	5
0	1	2	3	4

0	1	2	3	4

0	1	2	3	4



Topic : Queue

```
void Enqueue (int data){  
    if (rear == Max-1){  
        printf ("Queue is full");  
        return;  
    }  
    arr[++rear] = data; // rear=rear+1; arr[rear]=data  
    if (front == -1)  
        front = 0;  
}
```




Queue

Delete()

$f=0$
 $r=4$

1	2	3	4	5
0	1	2	3	4

$f=1$
 $r=4$

1	2	3	4	5
0	1	2	3	4

delete from front Index and increment the front

$f=1$
 $r=4$

	2	3	4	5
0	1	2	3	4

$f=1$
 $r=4$

	2	3	4	5
0	1	2	3	4

Insert()

overflow

Space available circularly



Queue

$f=1$
 $r=4$

Delete

	2	3	4	5
0	1	2	3	4

$f=2$
 $r=4$

	2	3	4	5
0	1	2	3	4

$f=2$
 $r=4$

Delete

		3	4	5
0	1	2	3	4

$f=3$
 $r=4$

		3	4	5
0	1	2	3	4



Queue

Delete

$$f = 4$$
$$r = 4$$



$$f = -1$$
$$r = -1$$



Before Increment front we need to check
if the Last element deleted ($front = rear$)

if yes : $front = rear = -1$

else $front = front + 1$



Queue

Delete

$f=3$
 $r=4$

			4	5
0	1	2	3	4

$f=4$
 $r=4$

			4	5
0	1	2	3	4

$f=0$
 $r=0$

Delete()

<u>1</u>				
0	1	2	3	4

$f=-1$
 $r=-1$

0	1	2	3	4



Topic : Queue

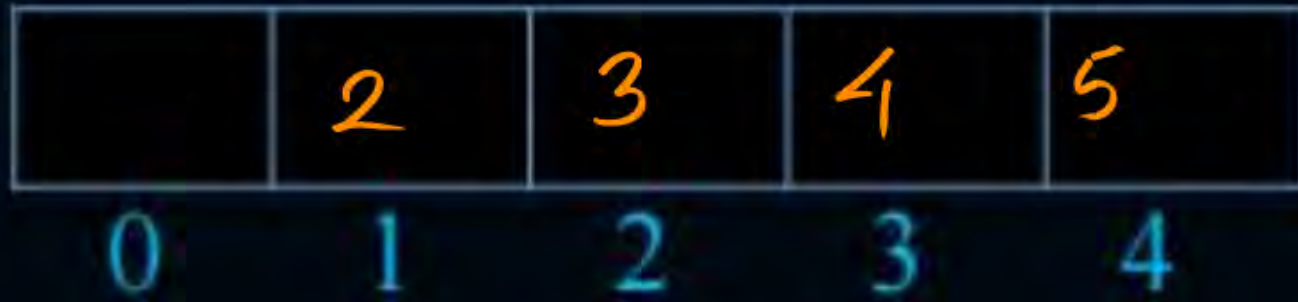


```
int dequeue() {  
    int data;  
    if (isEmpty()) {  
        printf("Queue is Empty");  
        return -1;  
    }  
    data = arr[front]; // arr[front] = 0;  
    if (front == rear)  
        front = rear = -1;  
    else  
        front++;  
    return data;  
}
```



Circular Queue

$f=1$
 $r=4$



Linear queue if rear Index reaches to $\text{max}-1$ it declared as full

Circular Queue: attach the end of queue

$$\begin{array}{l} \text{rear} = \text{rear} + 1 \\ \text{front} = \text{front} + 1 \end{array}$$

$$\text{rear} = (\text{rear} + 1) \% 5 \quad (\text{Max})$$

$$\text{front} = (\text{front} + 1) \% 5 \quad (\text{Max})$$



Circular Queue



$f=1$
 $r=4$

	2	3	4	5
0	1	2	3	4

$f=1$
 $r=0$

6	1	2	3	4
0	1	2	3	4

Insert(6)

if ($front == rear + 1$) first condition

$f=0$
 $r=4$

1	2	3	4	5
0	1	2	3	4

full
if ($front == 0 \ \&\& \ rear = Max - 1$)

Second condition

if $((front == rear + 1) \parallel (front == 0 \ \&\& \ rear == Max - 1))$

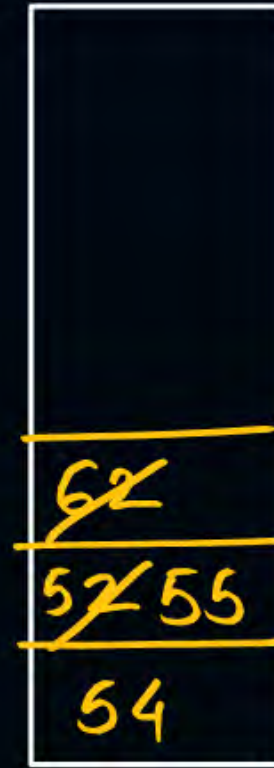
$front == (rear + 1) \% Max$



Topic: Question

Consider the following sequence of operations on an empty stack.

push (54); push (52); pop (); push (55); push (62); s = pop ();



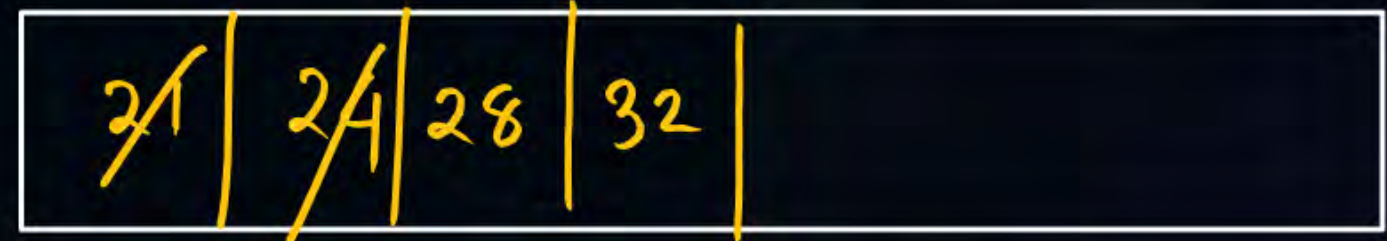
S = pop()

S = 62

Consider the following sequence of operations on an empty queue.

enqueue (21), enqueue (24), dequeue (); enqueue (28); enqueue (32); q = dequeue ();

The value of s+q is 62+24=86



q = dequeue()

q = 24

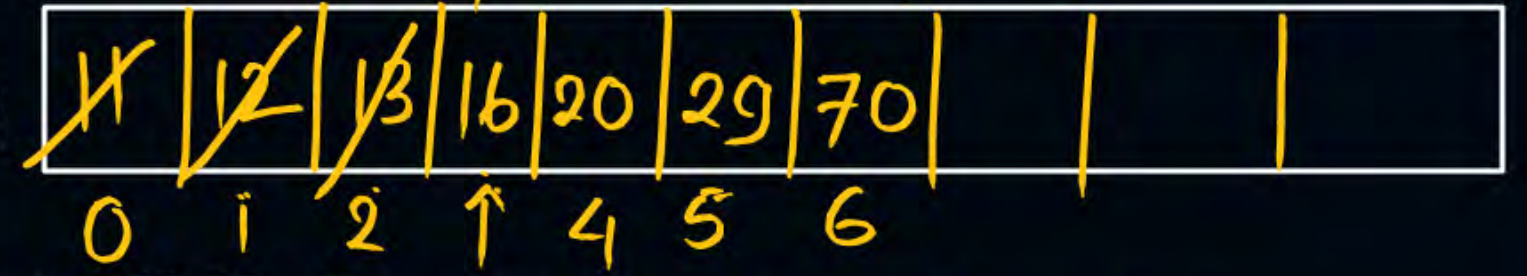


Question



If the following operation are performed on queue of size 10 $f=3$ $r=6$

I. Enqueue (11), Enqueue (12), Enqueue (13)



II. Dequeue(), Dequeue(), Enqueue(16), Enqueue(20), Enqueue (29), Enqueue (70), $x = \text{Dequeue}()$. If Y is Index of front & z is the Index of rear then $X+Y-Z$ is

$$13 + 3 - 6 = 10$$



Topic: Question

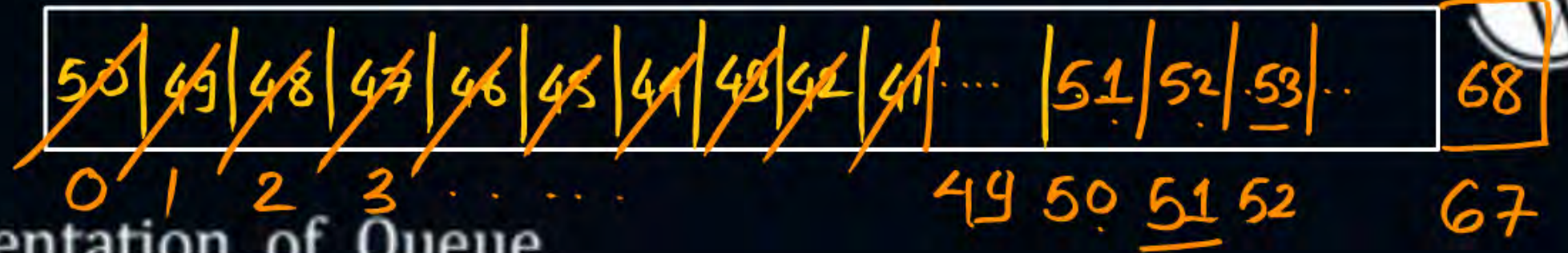


If the following operation are performed on queue of size 10

- I. Enqueue (11), Enqueue (12), Enqueue (13)
- II. Dequeue(), Dequeue(), Enqueue(16), Enqueue(20), Enqueue (29), Enqueue (70), $x = \text{Dequeue}()$. If Y is Index of front & z is the Index of rear then $X+Y-Z$ is ____10____



Topic: Question



#Q consider the array implementation of Queue,
array of size 100 With base address of 1000 and
size 2B. Suppose the following operation is
being performed:

$$1000 + 67 \times 2$$

$$= \boxed{1134}$$

- I. Enqueue 1 to 50 element in reverse order ✓
- II. Dequeue 10 elements from the queue
- III. Insert 30 Numbers 51 to 80 in the given order

The address of element 68 is ✓ 1134



Topic: Question

#Q consider the array implementation of Queue, array of size 100 With base address of 1000 and size 2B. Suppose the following operation is being performed:

- I. Enqueue 1 to 50 element in reverse order
- II. Dequeue 10 elements from the queue
- III. Insert 30 Numbers 51 to 80 in the given order

The address of element 68 is __1134__



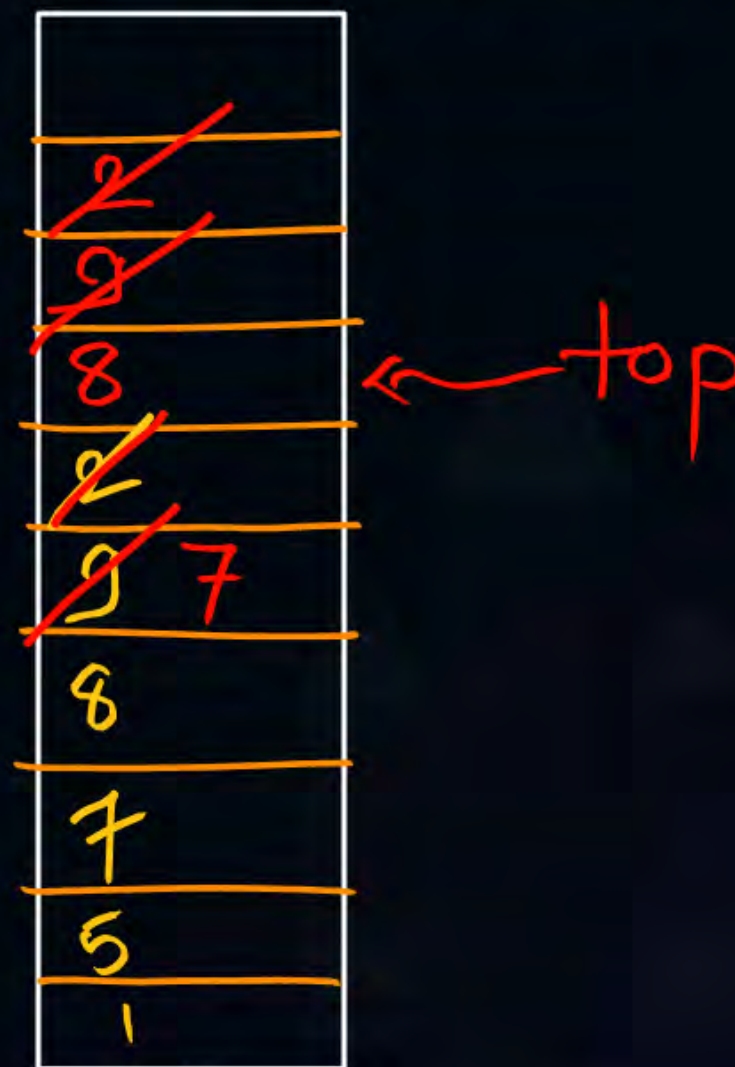
Topic: GATE 2023, 2-Marks



1	5	7	8	9	2		
--------------	--------------	--------------	--------------	--------------	--------------	--	--

Consider a sequence a of elements $a_0 = 1, a_1 = 5, a_2 = 7, a_3 = 8, a_4 = 9$, and $a_5 = 2$. The following operations are performed on a stack S and a queue Q , both of which are initially empty.

- I. push the elements of a from a_0 to a_5 in that order into S .
- II. enqueue the elements of a from a_0 to a_5 in that order into Q .
- III. pop an element from S .
- IV. dequeue an element from Q .
- V. pop an element from S .
- VI. dequeue an element from Q .
- VII. dequeue an element from Q and push the same element into S .
- VIII. Repeat operation VII three times. ✓
- IX. pop an element from S .
- X. pop an element from S .



The top element of S after executing the above operations is _____.

68
69

145

(282)

#Q. Consider Stack S1 and queue Q with initial configuration as
ENQ ENQ GO(S)

4	(Top)
3	4
2	5
1	4

4	5
5	4
3	2

Queue Q

push push GO(Q) ENQ(4) ENQ(5) Stack S1

GO(S₁) , push S₁

GO(Q) GO(S)

Only the following operations are available:

- 4 ENQToQ: Pop the top element from S1 and ENQUEUE it on Q. Time 17 Units
- 3 PushToS1: Dequeue element from Q and push it on S1. Time 23 Units
- 5 GenerateOutputS1 : Pop the top element from S1 and output it to the user. Time 29 Units
- GenerateOutputQ: Dequeue element from the Queue and produce output: Time 29 Units

If we want output 2, 3, 1, 5, 4 then total time required for this.

#Q. Consider Stack S1 and queue Q with initial configuration as

400 (Top)
3
2
1

Stack S1

5		
---	--	--

Queue Q

Only the following operations are available:

ENQToQ: Pop the top element from S1 and ENQUEUE it on Q. Time 17 Units

PushToS1: Dequeue element from Q and push it on S1. Time 23 Units

GenerateOutputS1 : Pop the top element from S1 and output it to the user.
Time 29 Units

GenerateOutputQ: Dequeue element from the Queue and produce output:
Time 29 Units

If we want output 2, 3, 1, 5, 4 then total time required for this. 282

THANK - YOU